

评分：_____



上海大学

SHANGHAI UNIVERSITY

课程论文

COURSE PAPER

论文题目

计算机网络综合实验二

ARP 欺骗

学 院 计算机工程与科学学院

专 业 计算机科学与技术

学 号 23121769

学生姓名 赵文字

ARP 欺骗原理及其实验验证

23121769 赵文字

计算机工程与科学学院

摘要：ARP 协议是局域网中实现 IP 地址与 MAC 地址映射的重要基础协议，但其在设计上缺乏有效的身份认证机制，存在一定的安全隐患。本文在可控的虚拟化实验环境中，利用 ARP 欺骗技术对靶机进行攻击实验，观察并分析了 ARP 缓存表变化及网络通信状态的影响。实验结果表明，攻击者可通过伪造 ARP 响应报文篡改靶机本地 ARP 表，从而干扰甚至阻断正常通信。通过对实验过程和结果的分析，进一步探讨了 ARP 欺骗的原理及其潜在安全风险，为理解局域网安全问题提供了实践依据。

关键字：ARP 协议，ARP 欺骗，局域网安全。

一、引言

随着局域网的广泛应用，地址解析协议（ARP）在主机间通信中发挥着重要作用。然而，由于 ARP 协议在设计上缺乏有效的身份认证机制，其在实际网络环境中存在一定的安全隐患。ARP 欺骗正是利用这一特性，通过伪造 ARP 响应信息，干扰主机对 IP 与 MAC 地址的正常映射关系，从而影响网络通信的正常进行。为加深对 ARP 协议工作原理及其安全问题的理解，本文在可控的实验环境下，对 ARP 欺骗相关技术进行了学习与实践，通过实验现象对其影响进行分析。

二、ARP 协议与 ARP 欺骗原理

ARP 协议，即 Address Resolution Protocol。在局域网中，网络是以“帧”的形式传输数据的。一台主机想要和另一台主机进行直接通信，必须要知道目标主机的 MAC 地址，目标主机的 MAC 地址理应就包含在数据帧中。显然在双方刚开始通信的时候，发送方是无从得知目标主机的 MAC 地址的，需要通过地址解析这个过程。所谓“地址解析”就是主机在发送帧前将目标 IP 地址转换成目标 MAC 地址的过程。ARP 协议的基本功能就是通过目标设备的 IP 地址，查询目标设备的 MAC 地址，以保证通信的顺利进行。

在实际工作过程中，ARP 协议通过发送 ARP 请求报文来询问目标 IP 地址对应的 MAC 地址，该请求通常以广播方式在局域网内发送。拥有对应 IP 地

址的主机会返回 ARP 响应报文，告知自身的 MAC 地址，发送方随后将该映射关系存入本地 ARP 缓存表，以减少后续通信中的查询开销。需要注意的是，ARP 协议在设计时并未对 ARP 响应的来源进行严格验证，主机往往默认信任接收到的 ARP 响应信息，并直接更新本地 ARP 表。

ARP 欺骗正是利用了上述机制上的缺陷。攻击者通过伪造 ARP 响应报文，将错误的 IP 与 MAC 地址映射关系发送给目标主机，使其误认为攻击者的 MAC 地址对应某一合法 IP 地址，从而导致 ARP 缓存表被篡改。在这种情况下，原本应发送给正常主机的数据可能被转发至攻击者，实现对通信过程的干扰甚至中间人攻击。

（比较重要的点，我一开始没怎么理解，后来和一个同学聊天的过程中才意识到）ARP 欺骗并不会改变网关设备在物理层面的 MAC 地址，而是通过伪造 ARP 响应报文，篡改被攻击主机本地 ARP 缓存表中 IP 与 MAC 地址的映射关系，使主机在逻辑层面上误将攻击者的 MAC 地址视为网关的 MAC 地址。

简单来说，就是 ARP 协议不会对于响应来源进行检验，所以一台主机可能会接收到非请求的主机可能接受非请求的 ARP 响应，如果攻击者伪造 IP - MAC 映射关系，那么就会导致 ARP 表被错误更新，从而被“欺骗”。

下图为一个简略版的原因说明。

正常通信：
[受害者电脑] <----> [路由器/网关] <----> [互联网]

ARP欺骗后：
[受害者电脑] <----> [攻击者电脑] <----> [路由器/网关] <----> [互联网]
(受害者以为攻击者就是网关)

图 1 ARP 欺骗原理

三、实验环境与实验方案设计

本次实验在可控的虚拟化环境中完成，以避免对真实网络造成影响。实验平台为一台 windows 系统的靶机（被攻击的电脑，是我本人的笔记本电脑）及一台在 workstation 安装的 ubuntu 系统的虚拟机，靶机作为被测试主机，虚拟机作为实验节点。虚拟机网络模式设置为桥接模式，与靶机一起连接到我手机的移动数据热点，以保证双方能够进行正常的 ARP 通信。（这里没有使用校园网的原因

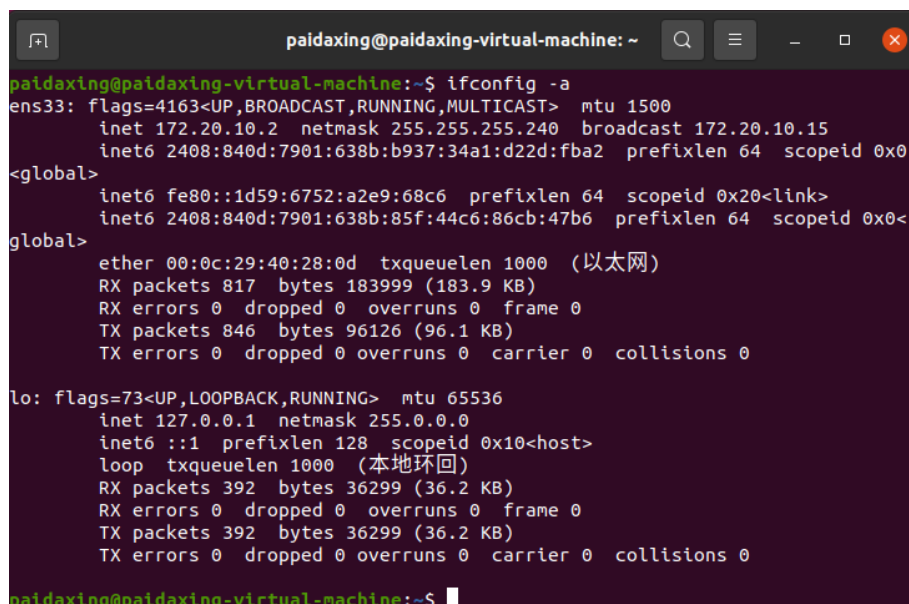
是，之前我做实验 1 还是 2 的时候感觉校园网有的时候不能被看作是一个很“单纯”的局域网，之前甚至出现过连在校园网的两台主机无法相互 ping 通的情况！
== 后面上网查阅了一下，发现好像校园网络通常采用多级网络隔离与安全防护机制，主机不一定处于同一二层广播域，可能影响 ARP 协议相关实验的正常进行，所以就换成手机热点了）。

实验开始前，首先确认各节点的 IP 地址与 MAC 地址信息，并记录初始的 ARP 缓存表状态。随后在虚拟机中运行 ARP 欺骗相关实验程序，对宿主机发送伪造的 ARP 响应数据包，观察宿主机 ARP 表的变化情况及网络通信状态。通过对比实验前后的网络行为，验证 ARP 欺骗对局域网通信的影响。

四、ARP 欺骗实验过程与结果

为了方便后文对于两台主机（攻击方：ubuntu 系统的虚拟机、靶机：windows 系统的本机）的称呼，这里规定一下对于两台主机的称呼分别为攻击主机与靶机。

首先，在确保了实验环境如同第三板块所讲的那样配置好后，先通过在攻击主机与靶机主机终端分别输入 `ifconfig -a` 以及 `ipconfig /all` 的指令，查看两台主机的 ip 地址以及网关等信息，具体信息如下图 2、图 3 所示：



```
paidaxing@paidaxing-virtual-machine: ~  
paidaxing@paidaxing-virtual-machine:~$ ifconfig -a  
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 172.20.10.2 netmask 255.255.255.240 broadcast 172.20.10.15  
    inet6 2408:840d:7901:638b:b937:34a1:d22d:fba2 prefixlen 64 scopeid 0x0  
<global>  
    inet6 fe80::1d59:6752:a2e9:68c6 prefixlen 64 scopeid 0x20<link>  
    inet6 2408:840d:7901:638b:85f:44c6:86cb:47b6 prefixlen 64 scopeid 0x0<  
global>  
    ether 00:0c:29:40:28:0d txqueuelen 1000 (以太网)  
    RX packets 817 bytes 183999 (183.9 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 846 bytes 96126 (96.1 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (本地环回)  
    RX packets 392 bytes 36299 (36.2 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 392 bytes 36299 (36.2 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
paidaxing@paidaxing-virtual-machine:~$
```

图 2 攻击主机 ip 地址

```
C:\Users\fdfx_>ipconfig

Windows IP 配置

未知适配器 本地连接:

    媒体状态 . . . . . : 媒体已断开连接
    连接特定的 DNS 后缀 . . . . . :

以太网适配器 以太网 3:

    媒体状态 . . . . . : 媒体已断开连接
    连接特定的 DNS 后缀 . . . . . :

无线局域网适配器 本地连接* 1:

    媒体状态 . . . . . : 媒体已断开连接
    连接特定的 DNS 后缀 . . . . . :

无线局域网适配器 本地连接* 2:

    媒体状态 . . . . . : 媒体已断开连接
    连接特定的 DNS 后缀 . . . . . :

无线局域网适配器 WLAN:

    连接特定的 DNS 后缀 . . . . . : shu.edu.cn
    IPv6 地址 . . . . . : 2408:840d:7901:638b:5edd:6be9:5993:80fd
    临时 IPv6 地址 . . . . . : 2408:840d:7901:638b:f9bb:27f:fed:c967
    本地链接 IPv6 地址 . . . . . : fe80::fc8e:dacc:72c:7bc7%9
    IPv4 地址 . . . . . : 172.20.10.4
    子网掩码 . . . . . : 255.255.255.240
    默认网关 . . . . . : fe80::2cc2:53ff:fea4:4764%9
                        172.20.10.1
```

图 3 靶机 ip 地址

在靶机终端中输入“ping 172.20.10.1”，可以查看网关的畅通与否。此时再次输入“ping www.baidu.com”，可以看到能 ping 通本网段的网关以及一些公网的 ip 地址。如下图所示：

```
C:\Users\fdfx_>ping www.baidu.com

正在 Ping www.a.shifen.com [153.3.238.28] 具有 32 字节的数据:
来自 153.3.238.28 的回复: 字节=32 时间=19ms TTL=52
来自 153.3.238.28 的回复: 字节=32 时间=48ms TTL=52
来自 153.3.238.28 的回复: 字节=32 时间=50ms TTL=52
来自 153.3.238.28 的回复: 字节=32 时间=87ms TTL=52

153.3.238.28 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 19ms, 最长 = 87ms, 平均 = 51ms

C:\Users\fdfx_>ping 172.20.10.2

正在 Ping 172.20.10.2 具有 32 字节的数据:
来自 172.20.10.2 的回复: 字节=32 时间<1ms TTL=64
来自 172.20.10.2 的回复: 字节=32 时间<1ms TTL=64
来自 172.20.10.2 的回复: 字节=32 时间<1ms TTL=64
来自 172.20.10.2 的回复: 字节=32 时间<1ms TTL=64

172.20.10.2 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms
```

图 4 攻击前连通性

我这里使用的是 ettercap 软件（ubuntu 系统上的）进行对于靶机的 ARP 欺骗攻击。在安装好 ettercap 软件包后，在攻击主机的终端中输入 ettercap -G 后，即可以运行该软件（没有启动超级管理员的原因是超级管理员会出现一些奇奇怪怪的 bug 所以就直接不用了）。该软件是有一个 gui 窗口的。

在打开了该软件后，点击右上角的运行按钮，选择当前网卡后，点击左上角的扫描按钮后，扫描该网段的所有主机的相关信息，显示主机列表，如下图所示：

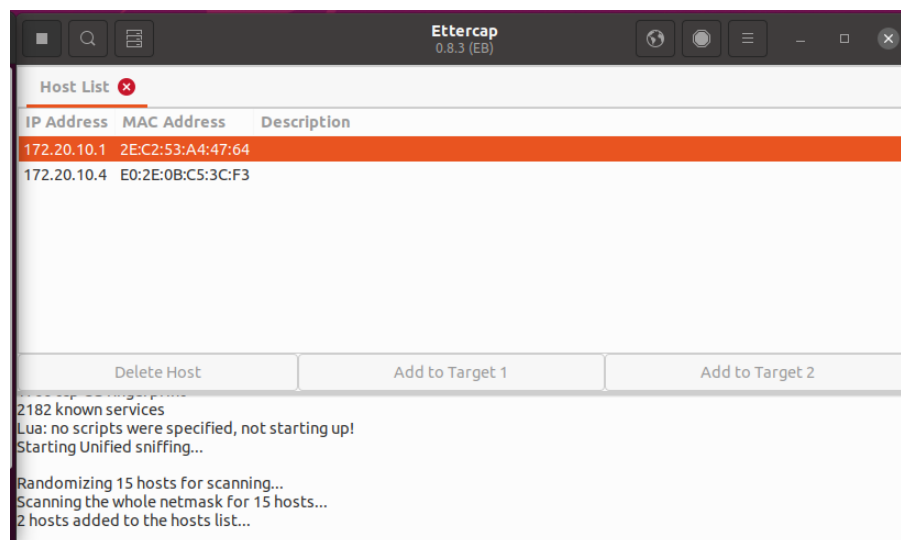


图 5 扫描到的主机列表

扫描完成后，发现有两个主机，一个就是靶机(172.20.10.4)，另一个是网关 172.20.10.1。选中 172.20.10.1，按“Add To Target 1”，加入目标 1；选中 172.20.10.4，按“Add To Target 2”，加入目标 2。点击右上角的三条横线中的 targets->current targets，即可查看确定目前确实已经将该两台机器添加为待欺骗攻击的对象了。

接着便可以进行 ARP 欺骗，按右上角的地球图标中的 Arp poisoning…，勾选 only poison one-way，点击 ok 开始进行欺骗。如下图所示：

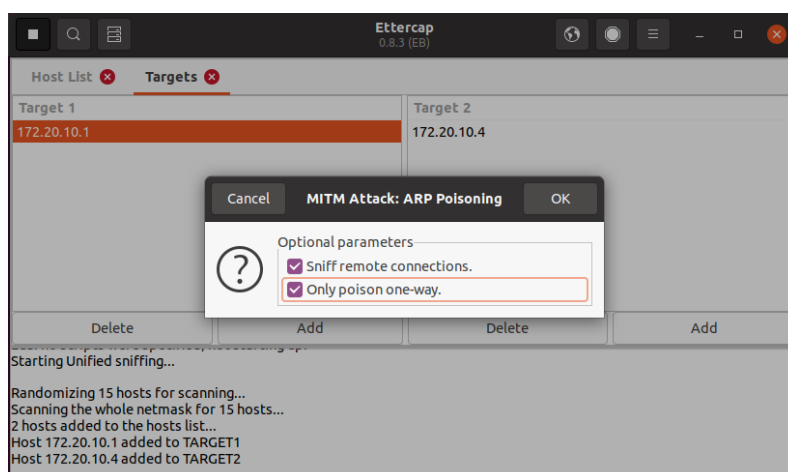


图6 其中一步

回到靶机的终端中，再依次输入“ping 172.20.10.1”和“ping www.baidu.com”，发现网关地址和百度网址都 ping 不通。

```
C:\Users\fdfx_>ping 172.20.10.1

正在 Ping 172.20.10.1 具有 32 字节的数据:
来自 59.79.4.209 的回复: 无法访问目标主机。
来自 59.79.4.209 的回复: 无法访问目标主机。
来自 59.79.4.209 的回复: 无法访问目标主机。
来自 59.79.4.209 的回复: 无法访问目标主机。

172.20.10.1 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),

C:\Users\fdfx_>ping 172.20.10.4

正在 Ping 172.20.10.4 具有 32 字节的数据:
来自 59.79.4.209 的回复: 无法访问目标主机。
来自 59.79.4.209 的回复: 无法访问目标主机。
来自 59.79.4.209 的回复: 无法访问目标主机。
来自 59.79.4.209 的回复: 无法访问目标主机。

172.20.10.4 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
```

图7 被攻击后 ping 不通

退出 ARP 欺骗攻击后，在靶机 cmd 中清理一下 ARP 缓存“arp -d”，再次测试查看靶机的网络情况，发现又可以 ping 通了。

除去 ettercap 这个软件，我还在 gpt 的帮助下编写了一个用于监测 arp-ip 表映射关系的 c++程序，实时监测 ARP 欺骗是否起效。

由于编写的代码篇幅较长，就不在此进行展示，具体内容可见附件。为了形成对比实验，在 ARP 欺骗前，先在攻击主机上监测一次。

输入 g++ -std=c++17 arp.cpp -o arp -lpcap -pthread -DHAVE_PCAP 进行编译后，再输入 sudo ./arp -t 172.20.10.1 -i ens33 -n 3 -l arp.log 运行该程序，在靶机上 ping 一下 172.20.10.1（网关），可以发现这边捕捉到了 arp-ip 表，此时其与网关的 MAC 地址还是正确的，未被改变，如下图所示：

```
paidaxing@paidaxing-virtual-machine:~/文档/system learning$ sudo ./arp -t 172.20.10.1 -i ens33 -n 3 -l arp.log
[sudo] paidaxing 的密码:
Starting ARP monitor. Targets: 172.20.10.1 | pcap iface: ens33 | interval: 3s
[2026-01-06 13:18:53] OBSERVED ARP: 10.88.64.1 -> 80:05:88:3f:e1:1e
[2026-01-06 13:18:54] OBSERVED ARP: 10.88.64.1 -> 80:05:88:3f:e1:1e
[2026-01-06 13:18:54] OBSERVED ARP: 10.88.64.1 -> 80:05:88:3f:e1:1e
[2026-01-06 13:18:54] OBSERVED ARP: 10.88.64.1 -> 80:05:88:3f:e1:1e
[2026-01-06 13:18:55] OBSERVED ARP: 10.88.64.1 -> 80:05:88:3f:e1:1e
[2026-01-06 13:18:55] OBSERVED ARP: 172.20.10.2 -> 00:0c:29:40:28:0d
[2026-01-06 13:18:56] OBSERVED ARP: 172.20.10.2 -> 00:0c:29:40:28:0d
```

图8 被攻击前 arp-ip 表正常

而在被攻击后，再输入这行指令进行捕捉，发现此时与网关的 MAC 地址已被“更改”为了攻击主机的 MAC 地址，说明 ARP 欺骗成功。

```
paidaxing@paidaxing-virtual-machine:~/文档/system learning$ sudo ./arp -t 172.20.10.1 -i ens33 -n 3 -l arp.log
Starting ARP monitor. Targets: 172.20.10.1 | pcap iface: ens33 | interval: 3s
[2026-01-06 13:08:24] OBSERVED ARP: 172.20.10.2 -> 00:0c:29:40:28:0d
[2026-01-06 13:08:24] OBSERVED ARP: 172.20.10.1 -> 2e:c2:53:a4:47:64
[2026-01-06 13:08:31] OBSERVED ARP: 172.20.10.4 -> 00:0c:29:40:28:0d
[2026-01-06 13:08:38] OBSERVED ARP: 172.20.10.2 -> 00:0c:29:40:28:0d
[2026-01-06 13:08:38] OBSERVED ARP: 172.20.10.4 -> e0:2e:0b:c5:3c:f3
[2026-01-06 13:08:40] OBSERVED ARP: 172.20.10.4 -> 00:0c:29:40:28:0d
[2026-01-06 13:08:51] OBSERVED ARP: 172.20.10.4 -> 00:0c:29:40:28:0d
[2026-01-06 13:09:01] OBSERVED ARP: 172.20.10.4 -> 00:0c:29:40:28:0d
[2026-01-06 13:09:04] OBSERVED ARP: 172.20.10.4 -> e0:2e:0b:c5:3c:f3
[2026-01-06 13:09:05] OBSERVED ARP: 172.20.10.4 -> e0:2e:0b:c5:3c:f3
[2026-01-06 13:09:06] OBSERVED ARP: 172.20.10.4 -> e0:2e:0b:c5:3c:f3
```

图 9 被攻击后 arp-ip 表异常

（做完后的反思）做完这个实验后，询问了老师，发现自己忽略了要自己编写程序进行 ARP 欺骗，而不是直接用已经实现好的软件，所以又紧急增加了以下内容……

新写了一个 c 语言程序，命名为 arp2.c，其作用就是先扫描出攻击主机的所有网络接口，选择要使用的网络接口后，构造并发送伪造的 ARP 响应包，让靶机错误地更新其 ARP 缓存，将特定 IP 地址映射到错误的 MAC 地址，实现 ARP 的欺骗，具体程序见附件中。运行结果如下所示，如同前一种方式，这个 c 语言程序也成功的进行了 ARP 欺骗，在欺骗期间，靶机无法 ping 通网卡以及外界设备：

```
paidaxing@paidaxing-virtual-machine: ~/文档/system learning
paidaxing@paidaxing-virtual-machine:~/文档/system learning$ sudo ./arp2
[sudo] paidaxing 的密码:
1. ens33 (No description available)
2. lo (No description available)
3. any (Pseudo-device that captures on all interfaces)
4. bluetooth-monitor (Bluetooth Linux Monitor)
5. nflog (Linux netfilter log (NFLOG) interface)
6. nfqueue (Linux netfilter queue (NFQUEUE) interface)
Enter the interface number (1-6):1

=====
本地MAC: 12:34:56:78:9a:ff
本地IP: 172.20.10.2
目标MAC: 2e:c2:53:a4:47:64
目标IP: 172.20.10.4
伪造MAC: aa:bb:cc:dd:ee:ff
伪造IP: 172.20.10.6
=====

第 1 个ARP包发送成功
第 2 个ARP包发送成功
第 3 个ARP包发送成功
```

图 10 运行过程中输出内容

五、实验分析与安全思考

通过本次在可控虚拟化环境中的实测，我们能够直观地把理论与现象对应起来：攻击主机持续发送伪造的 ARP 响应后，靶机本地 ARP 缓存中“网关 IP → MAC”的映射被错误替换，导致靶机向网关发送的数据被转发到攻击主机，网络连通性被破坏并在清除缓存后恢复。这个过程清晰地证明了 ARP 协议本身缺乏对响应来源的验证——主机只要收到 ARP 响应，通常就会信任并更新本地表项，从而为攻击者创造了可乘之机（实验过程与结果见文中描述与截图）。

从攻击效果看，本实验使用的单向欺骗已经足以造成拒绝服务（目标主机无法访问网关或互联网）；若升级为双向欺骗并配合数据转发/流量转发（即真正的中间人攻击），就可以实现对会话的监听与篡改，威胁等级会显著上升。值得注意的是，ARP 欺骗并不会在物理层改写任何设备的真实 MAC，它只是篡改了受害主机的逻辑层（ARP 缓存）映射——这也解释了为什么停止攻击并清空缓存后通信能立即恢复。

从防护角度看，单一手段难以完全杜绝 ARP 欺骗，应采取多层次防护策略：在接入层交换机上启用动态 ARP 检测或 DHCP Snooping、配置端口安全（port security）限制 MAC 绑定、对关键主机采用静态 ARP 表（谨慎使用）、并尽量在应用层使用端到端加密协议（HTTPS、SSH、TLS 等）来降低中间人带来的信息泄露风险。此外，部署实时监测系统（如本次自行编写的 ARP 监测程序）可以在攻击早期发现异常映射并快速响应——这些检测程序在教学与运维中都非常有价值，但又必须避免被误用，确保只在获得授权的测试网络中运行。

总体而言，本次实验不仅加深了我对 ARP 协议工作机制的理解，也使笔者直观认识到传统网络协议在安全性方面的局限性。通过实验现象与分析相结合，有助于在今后的学习和实践中更加重视网络安全问题，从协议设计与实际部署两个层面进行综合考虑。